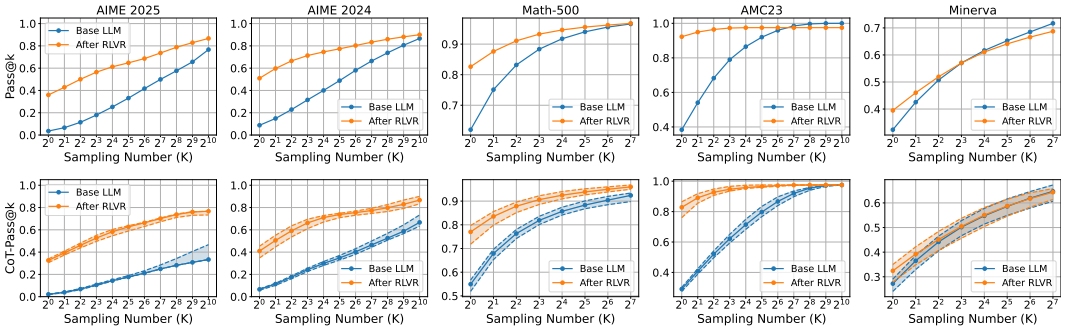

Figure 2: Comparisons of Pass@K (the top row) and CoT-Pass@K (the bottom row) on five math benchmarks (different columns) to show how RLVR could improve base LLMs. Here the base LLM is Qwen2.5-32B, and the post-RLVR model is DAPO-Qwen-32B. For CoT-Pass@K, we perform multiple verifications for each CoT using DeepSeek-R1-0528-Qwen3-8B, and display the results determined by *any-correct*, *all-correct*, and *majority-correct* strategies, which constitute the shaded area in lower subplots.

gap on the LiveCodeBench v6 (Jain et al., 2025). In this work, we extend the Pass@K experiments of Chen et al. (2025b) to more LiveCodeBench versions and include another similar open-source study (He et al., 2025) starting RLVR from distilled LLMs as a comparison. All these results clearly disclose the extended reasoning boundary of distilled LLMs on competitive coding after RLVR.

The Importance of Correct CoTs Recent studies have also highlighted the importance of verifying the correctness of CoTs (Arcuschin et al., 2025; McGinness & Baumgartner, 2025; Shojaee et al., 2025). However, their approaches focus on defining synthetic reasoning tasks where the correctness of reasoning CoTs can be verified easily. While this is an interesting and effective approach for fully examining reasoning correctness, it is difficult to apply to unstructured reasoning scenarios, such as in math and code. In this work, we argue that the LLM-as-a-CoT-Judge paradigm could play a crucial role in more general reasoning tasks, and emphasize the pressing need for the design of evaluation benchmarks to assess the reliability of emerging LLM verifiers.

3 Extended Reasoning Capability Boundary After RLVR

In this section, we present concrete benchmark evaluations that demonstrate how RLVR can fundamentally enhance the reasoning abilities of LLMs. This enhancement goes beyond mere improvements in sampling efficiency; it also expands the reasoning capability boundary. However, to effectively observe this enhancement, it is crucial to adopt an appropriate RLVR training recipe, select challenging benchmarks that are free from data contamination, and utilize reliable evaluation metrics. Without these measures, one might only observe improvements in sampling efficiency, with no actual change in reasoning capacity. Below, we discuss two representative cases from both the math and code domains, showcasing genuinely extended reasoning boundaries.

3.1 Math Reasoning

We begin by revisiting the Pass@K experiments conducted on the open-source model, DAPO-Qwen-32B (Yu et al., 2025), which successfully reproduced R1-Zero (Guo et al., 2025) using the base LLM, Qwen2.5-32B (Qwen, 2024), and a curated set of $17k$ mathematical problems. A key contribution of our work is the introduction of a novel evaluation metric, CoT-Pass@K, which emphasizes the importance of evaluating the correctness of detailed reasoning steps for mathematical questions, rather than relying solely on answer correctness.

However, precisely measuring the correctness of CoTs at scale is inherently challenging due to the unstructured, lengthy, and complex nature of math CoTs. Fortunately, we can leverage specialized open-source LLMs, such as DeepSeek-R1-0528-Qwen3-8B (DeepSeek, 2025), as a powerful yet lightweight verifier, employing an LLM-as-a-CoT-Judge paradigm. In this study, we use this

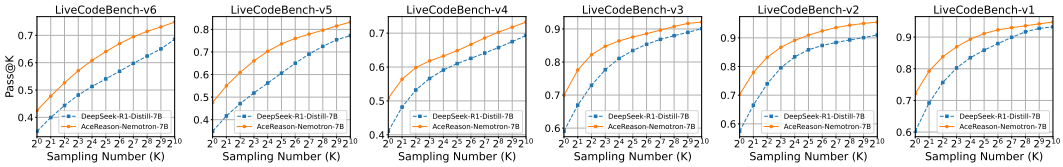

Figure 3: Comparisons of Pass@K across six LiveCodeBench versions to show how much RLVR could enhance distilled LLMs. Here the distilled LLM is DeepSeek-R1-Distill-Qwen-7B, and the post-RLVR model is AceReason-Nemotron-7B.

verifier multiple times for each reasoning CoT generated by DAPO-Qwen-32B and Qwen2.5-32B, employing three distinct strategies to assess CoT correctness: *any-correct* (at least one verification returns correct), *all-correct* (all verifications must return correct), and *majority-correct* (the majority vote determines the outcome). To further ensure the reliability of these verifications, we manually inspect cases where the Pass@K metric yields a small positive value, but the CoT-Pass@K metric returns zero. The details of our LLM-as-a-CoT-Judge approach can be found in Appendix A.3.

Figure 2 presents a comprehensive comparison using both Pass@K and CoT-Pass@K metrics across prominent math-reasoning benchmarks. As shown in the top row, the Pass@K results align with the observations in (Yue et al., 2025): the performance of the base LLM quickly catches up with and even surpasses the post-RLVR model as K increases. However, in stark contrast, the CoT-Pass@K results on AIME 2024 and AIME 2025 reveal a consistent and significant performance gap between the models across all values of K (up to 1024). This gap is particularly pronounced on AIME 2025, possibly due to its complete absence of unintentional data contamination, as it was released after the base model's training cutoff. We perform manual inspections to ensure the distinct gaps observed using the CoT-Pass@K metric are reliable (see examples in Appendix A.8.1 and A.8.2). Our LLM verifier effectively identifies critical errors, which we agree should be rejected. These results demonstrate the extended reasoning boundary of DAPO-Qwen-32B over Qwen2.5-32B.

Additionally, we observe that on other benchmarks such as MATH-500 and AMC23, the effects of RLVR seem less pronounced, as the base LLM is already capable of solving these problems correctly with sufficient attempts. This may be due to 1) the problems being simple enough for the base LLM to solve using its existing knowledge, or 2) the problems being part of its pre-training data, allowing the base LLM to recall the correct solution after multiple trials. Distinguishing between these possibilities is challenging without knowing the exact training data used for Qwen2.5-32B. Furthermore, on the Minerva benchmark, the post-RLVR model shows no improvement, likely due to a train-test domain mismatch. Minerva contains numerous physics problems and free-form answers, while the DAPO training data was restricted to math problems with integer answers. These results do not undermine the effectiveness of RLVR; rather, they highlight the importance of selecting appropriate benchmarks for evaluating RLVR progress.

3.2 Code Reasoning

Unlike RLVR for mathematical problems, where the correctness of extracted answer tokens is used as a proxy for passing, code reasoning relies on the actual execution of generated code snippets to verify correctness, significantly reducing the likelihood of guessing. Therefore, Pass@K serves as a reliable evaluation metric for code reasoning tasks.

In this section, we reproduce the Pass@K experiments across different versions of LiveCodeBench (Jain et al., 2025) to compare the performance of AceReason-Nemotron-7B (Chen et al., 2025b) with its pre-RLVR counterpart, DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025). As shown in Figure 3, we observe that AceReason-Nemotron-7B exhibits clear Pass@K improvements over DeepSeek-R1-Distill-Qwen-7B on most benchmark versions, even though the latter is a distillation model already demonstrating remarkable reasoning capabilities. These results suggest that even for distillation models, a high-quality RLVR training recipe can significantly extend the reasoning capability boundary, particularly for competitive coding tasks.

To further confirm the success of RLVR in extending reasoning boundaries for coding tasks, we evaluate another post-RLVR model, Skywork-OR1 (He et al., 2025), which has a fully reproducible

training recipe publicly available. Detailed results on LiveCodeBench-v6 can be found in Appendix A.4, where we observe a consistent Pass@K gap between Skywork-OR1 and DeepSeek-R1-Distill-Qwen-7B. Specifically, we find that only medium and hard problems in LiveCodeBench-v6 contribute to the differentiation between these two models for large K values, underscoring the importance of selecting challenging benchmarks.

4 A Theoretical Understanding of RLVR for LLMs

In addition to empirical evidences, we provide a theoretical understanding of how RLVR, as implemented in the GRPO algorithm (Shao et al., 2024), fundamentally incentivizes correct reasoning for pre-trained language models. We note a key distinction between RLVR for LLMs and traditional RL for randomly initialized models. Pre-trained LLMs, owing to their powerful likelihood estimation capabilities obtained during pre-training, can generate various CoTs and then produce possible answers. Some of them could coincidentally arrive at the ground truth, especially when the ground truth is in a simple format and can be easily guessed. In contrast, traditional RL simply optimizes for action trajectories that yield high rewards, without necessarily verifying the intrinsic correctness of each action along the path. For instance, in the Go game (Silver et al., 2017), every action is valid once the simulation environment is setup correctly. In the following, we start our theoretical analysis from a formal problem setup distinguishing CoT and answer tokens in LLM responses.

Problem Setup Given a question prompt q , we sample G responses $\mathbf{Y} = \{y_1, y_2, \ldots, y_G\}$ from policy π_θ, where π_θ is a LLM model parameterized by θ. Let c_i be the CoT in response y_i, and a_i the final answer. We define the following correctness indicators:

$$\mathcal{I}_{\text{CoT}}(c_i) = \begin{cases} 1 & \text{if } c_i \text{ is correct} \\ 0 & \text{otherwise} \end{cases}, \quad \mathcal{I}_{\text{Ans}}(a_i) = \begin{cases} 1 & \text{if } a_i \text{ is correct} \\ 0 & \text{otherwise} \end{cases}. \tag{1}$$

In this study, we define the CoT correctness $\mathcal{I}_{\text{CoT}}(c_i)$ as the intermediate tokens of a response (c_i) expressing necessary and accurate logics that lead to the ground truth. We use $p_c^\theta = P_{\pi_\theta}(\mathcal{I}_{\text{CoT}}(c) = 1)$ to denote the probability of generating a correct CoT. In practice, it is rather challenging to verify the CoT correctness because it is inherently unstructured, knowledge-intensive, and full of details. In contrast, the answer correctness $\mathcal{I}_{\text{Ans}}(a_i)$ is assumed to be verified programmatically. So we have a verifiable reward $R(y_i)$ that is binary and determined solely by answer correctness: $R(y_i) = \mathcal{I}_{\text{Ans}}(a_i)$. We calculate the standard GRPO advantage $\hat{A}(y_i)$ as:

$$\hat{A}(y_i) = \frac{R(y_i) - \mu_{\mathbf{Y}}}{\sigma_{\mathbf{Y}}}, \quad \mu_{\mathbf{Y}} = \frac{1}{G}\sum_{j=1}^{G} R(y_j), \quad \sigma_{\mathbf{Y}} = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(R(y_j) - \mu_{\mathbf{Y}})^2}. \tag{2}$$

Without loss of generality, we consider a policy gradient (Sutton et al., 1999) update:

$$\nabla_\theta J(\theta) \approx \frac{1}{G}\sum_{i=1}^{G} \hat{A}(y_i)\nabla_\theta \log \pi_\theta(y_i \mid q). \tag{3}$$

Assumptions Given the problem setup decoupling CoT and answer correctness, we introduce a critical *Logic Prior* assumption: compared with incorrect CoTs, correct CoTs have higher probabilities to induce correct answers. Thus we have

$$P(\mathcal{I}_{\text{Ans}}(a_i) = 1 \mid \mathcal{I}_{\text{CoT}}(c_i) = 1) = \alpha > P(\mathcal{I}_{\text{Ans}}(a_i) = 1 \mid \mathcal{I}_{\text{CoT}}(c_i) = 0) = \beta. \tag{4}$$

This assumption is based on the belief that pre-trained LLMs have established strong knowledge and logic priors. Besides, we also assume a learnable group ($\sigma_{\mathbf{Y}} > 0$) and a sufficiently large sampling number G to ensure stable gradient updates. Then, we establish the following theorem.

Theorem 1 (GRPO Implicitly Incentivizes Correct Reasoning) *For any prompt q satisfying our assumptions, the expected GRPO advantage $\mathbb{E}[\hat{A}(y_i)]$ satisfies:*

$$\mathbb{E}\left[\hat{A}(y_i) \mid \mathcal{I}_{CoT}(c_i) = 1\right] > 0, \quad \mathbb{E}\left[\hat{A}(y_i) \mid \mathcal{I}_{CoT}(c_i) = 0\right] < 0, \tag{5}$$

where $\hat{A}(y_i)$ is defined in equation 2. The GRPO policy gradient, as defined in equation 3, increase the probability of generating correct CoTs (p_c^θ) in the next round, so p_c^θ increases monotonically.